

```
"""
```

```
TIPE - Comptage automatique des points au basketball  
Simulation de la trajectoire d un projectile et validation du critere de  
panier.
```

```
Theme : Boucles, Cycles et Retroaction
```

```
"""
```

```
import numpy as np  
import matplotlib.pyplot as plt  
import matplotlib.patches as patches
```

```
# Parametres physiques
```

```
g = 9.81 # m/s2
```

```
H_panier = 3.05 # hauteur de l anneau (m)
```

```
R_anneau = 0.225 # rayon de l anneau (m)
```

```
X_panier = 6.75 # distance horizontale tir->panier (tir a 3 pts)
```

```
y0 = 2.0 # hauteur de lancer (m)
```

```
THETA_MIN = 33.0 # angle minimum d entree (degres) - critere Brancazio
```

```
def valider_panier(v0, alpha_deg):
```

```
    """
```

```
    Retourne (valide, x_passage, angle_entree).
```

```
    Critere 1 : le ballon passe dans la zone de l anneau.
```

```
    Critere 2 : angle d entree suffisant (>= THETA_MIN).
```

```
    """
```

```
    a = np.radians(alpha_deg)
```

```
    vx = v0 * np.cos(a)
```

```
    vy = v0 * np.sin(a)
```

```
    # Resolution de  $y_0 + v_y t - 0.5 g t^2 = H_{\text{panier}}$ 
```

```
    A = -0.5 * g
```

```
    B = vy
```

```
    C = y0 - H_panier
```

```
    disc = B**2 - 4*A*C if disc < 0:
```

```
    return False, None, None
```

```
    ts = [(-B + np.sqrt(disc)) / (2*A), (-B - np.sqrt(disc)) / (2*A)]
```

```
    ts = [t for t in ts if t > 0]
```

```
    if not ts:
```

```
    return False, None, None
```

```
    t_p = max(ts) # phase de descente
```

```
    xp = vx * t_p
```

```
    if abs(xp - X_panier) > R_anneau:
```

```
    return False, xp, None
```

```
    vy_p = vy - g * t_p
```

```
    theta = abs(np.degrees(np.arctan2(abs(vy_p), vx)))
```

```
    if theta < THETA_MIN:
```

```
    return False, xp, theta
```

```
    return True, xp, theta
```

```
def trajectoire(v0, alpha_deg, dt=0.001):
```

```
    a = np.radians(alpha_deg)
```

```
    vx = v0 * np.cos(a)
```

```
    vy = v0 * np.sin(a)
```

```
    ts = np.arange(0, 3, dt)
```

```
    xs = vx * ts
```

```

ys = y0 + vy * ts - 0.5 * g * ts**2
mask = ys >= 0
return xs[mask], ys[mask]

fig, axes = plt.subplots(1, 2, figsize=(14, 6))
fig.suptitle("Simulation TIPE - Trajectoires et zone de tir valide",
fontsize=14, fontweight="bold")

# Figure 1 : exemples de trajectoires
ax = axes[0]
ax.set_title("Exemples de trajectoires (X_panier = 6.75 m)") configs = [
    (9.0, 55, "Panier valide (55 deg, 9 m/s)",),
    (9.5, 50, "Panier valide (50 deg, 9.5 m/s)",),
    (7.5, 50, "Trop court",),
    (10.5, 38, "Angle insuffisant",),
    (9.0, 72, "Trop haut",),
]
couleurs = ["#1D9E75", "#0F6E56", "#E24B4A", "#BA7517", "#888780"]
for (v0, alpha, label), col in zip(configs, couleurs):
    xs, ys = trajectoire(v0, alpha)
    valide, _, _ = valider_panier(v0, alpha)
    ax.plot(xs, ys, "-" if valide else "--", color=col, lw=2 if valide else 1.2,
label=label)

ax.add_patch(patches.FancyArrowPatch(
    (X_panier - R_anneau, H_panier), (X_panier + R_anneau, H_panier),
    arrowstyle="-", color="black", lw=4))
ax.add_patch(patches.Rectangle((X_panier + R_anneau, H_panier), 0.05, 1.2,
color="gray", alpha=0.6)) ax.annotate("Anneau", xy=(X_panier, H_panier),
xytext=(X_panier + 0.5, H_panier + 0.5), fontsize=9, arrowprops=dict
(arrowstyle="->", color="black"))
ax.axhline(0, color="#ccc", lw=1)
ax.set_xlim(-0.3, 9); ax.set_ylim(0, 7) ax.set_xlabel("Distance horizontale
(m)"); ax.set_ylabel("Hauteur (m)") ax.legend(fontsize=8); ax.grid(True,
alpha=0.3)

# Figure 2 : carte de la zone de tir valide
ax2 = axes[1]
v0_vals = np.linspace(7, 12, 100)
alpha_vals = np.linspace(30, 75, 100)
Z = np.zeros((len(alpha_vals), len(v0_vals)))
for i, alpha in enumerate(alpha_vals):
    for j, v0 in enumerate(v0_vals):
        valide, _, _ = valider_panier(v0, alpha)
        Z[i, j] = 1 if valide else 0

ax2.contourf(v0_vals, alpha_vals, Z, levels=[-0.5, 0.5, 1.5], colors=
["#E6F1FB", "#9FE1CB"], alpha=0.9) ax2.contour(v0_vals, alpha_vals, Z, levels=
[0.5], colors="#0F6E56", linewidths=2) ax2.set_xlabel("Vitesse initiale v0
(m/s)"); ax2.set_ylabel("Angle de tir alpha (deg)") ax2.set_title("Zone de
tir valide (vert = panier compte)")
ax2.grid(True, alpha=0.3)

plt.tight_layout()

```

```
plt.savefig("/mnt/user-data/outputs/simulation_basketball.png", dpi=150,  
bbox_inches="tight")
```

```
# Affichage tableau de resultats
```

```
print("\nâ€œ Resultats de validation â€œ") print(f"{'v0 (m/s)':<10}  
{'alpha (deg)':<13}{'Valide':<9}{'x_passage (m)':<16}{'theta (deg)'}") print  
("-" * 58)
```

```
tests = [(9.0,55),(9.5,50),(7.5,50),(10.5,38),(9.0,72),(8.8,52)]
```

```
for v0, alpha in tests:
```

```
    valide, xp, theta = valider_panier(v0, alpha)
```

```
    xp_s = f"{xp:.3f}" if xp is not None else "---"
```

```
    theta_s = f"{theta:.1f}" if theta is not None else "---" print(f"{v0:<10}  
{alpha:<13}{'OUI' if valide else 'non':<9}{xp_s:<16}{theta_s}")
```